L7.1 — Self-Reading: UML Diagram Types

AICE2005 | Self-Study

University of Southampton

This session is self-reading

No live lecture for L7.1 — work through these notes at your own pace before the week 8 workshop.

- UML Cheat Sheet — quick-reference summary of the notation covered below
- Optional: LinkedIn Learning course — Software Design: Modeling with UML — not assessed, for anyone who wants a video walkthrough alongside these notes

© University of Southampton

UML — The 14 Diagram Types

> UML diagrams split into two families: **structure diagrams** (what the system is made of — static) and **behavior diagrams** (what the system does — dynamic).

0. Why UML, and why so many diagrams?

UML (Unified Modeling Language) is a standardized set of diagram notations for defining, visualizing, and documenting the artifacts of a software system — and sometimes a business model. It gives object-oriented teams a shared vocabulary to communicate designs, explore alternatives, and validate architecture before writing code.

There isn't one "UML diagram" because no single view can serve everyone on a project — an architect, a coder, a tester, and a technical writer each need a different angle and level of detail on the same system. UML covers that by splitting into 14 diagrams across two families:

Family	Diagrams
Structure (7)	Class, Object, Component, Deployment, Package, Composite Structure, Profile
Behavior (7)	Use Case, Activity, State Machine, Sequence, Communication, Interaction Overview, Timing

Structure Diagrams

1. Class Diagram

The central modeling technique in object-oriented design. Describes the **types of objects** in a system and the static relationships between them. A class box has three compartments: **name** (top, the only mandatory part), **attributes** (middle, `type` after the colon), **methods** (bottom, return type after the colon).

Visibility modifiers on attributes/methods:

Symbol	Visibility
`+`	Public
`-`	Private
`#`	Protected
`~`	Package

Method parameter direction: `in` (used, not changed by the method), `inout` (used and changed), `out` (purely a return channel).

© University of Southampton

2. Object Diagram

An **instance** of a class diagram: a snapshot of concrete objects, their attribute values, and the links between them at one point in time. Limited but useful scope — mainly for illustrating data-structure examples or validating a class diagram against a test case. Object names are written `objectName : ClassName`, underlined.

```
flowchart LR
    r["r : Robot<br/>state = moving"] --> w["w : World"]
    w --> a1["a1 : Area"]
    w --> a2["a2 : Area"]
    a2 --> wall1["wall1 : Wall<br/>width = 2m"]
    a2 --> wall2["wall2 : Wall<br/>width = 3m"]
    a2 --> wall3["wall3 : Wall<br/>width = 2m"]
    a2 --> door1["door1 : Door"]
    wall1 --- wall2
    wall2 --- wall3
```

A robot's world-view: it has identified an enclosed area with three walls and a door.

© University of Southampton

3. Component Diagram

Shows how components are wired together into larger components or systems — the architecture of components and their dependencies, including runtime, executable, and source-code components. Each component exposes a **provided interface** (lollipop symbol — services it offers) and/or a **required interface** (socket symbol — services it needs) on its ports.

```
flowchart LR
    PaymentGateway(["PaymentGateway<br/>«required»"]) --> OrderService
    subgraph OrderService["«component» OrderService"]
        direction LR
    end
    OrderService --> OrderAPI(["OrderAPI<br/>«provided»"])
```

A **subsystem** is a special kind of component — same notation, marked `«subsystem»` instead of `«component»`.

© University of Southampton

4. Deployment Diagram

Models the **physical** aspect of a system: hardware nodes and the software artifacts deployed on them. A 3D box represents a node (hardware or software); lines between nodes represent communication paths, optionally stereotyped with the protocol.

```
flowchart TD
    Client["«device»<br/>Client PC"] -->|HTTP| WebServer["«node»<br/>Web Server"]
    WebServer -->|SQL| DBServer["«node»<br/>Database Server"]
```

Deployment planning checklist (condensed from the full list): installation procedure and failure points, backups and rollback, data conversion, version-compatibility during rollout, staff/user training and documentation, integration with existing systems, reliability/failover equipment, and post-deployment security (firewalls, physical access).

© University of Southampton

5. Package Diagram

Groups related model elements into **packages** (UML's folders) to manage scope and show dependencies between subsystems/modules — e.g. a layered application. Packages can nest. One package depends on another (dashed arrow) if a change in the second can force a change in the first.

Element visibility inside a package:

Symbol	Visibility	Visible to
`+`	Public	Everyone who can see the namespace
`#`	Protected	Elements related by generalization
`~`	Package	Elements inside the same package
`-`	Private	Nothing outside the owning namespace

Dependency stereotypes: `«use»` (default — client uses supplier's public elements by name), `«import»` (supplier's public elements become public in the client), `«access»` (supplier's public elements become private in the client), `«trace»` (historical link between model versions, rare in practice).

© University of Southampton


```
flowchart LR
```

6. Composite Structure Diagram

Zooms inside a single class (or collaboration) to show its internal **parts**, **ports**, and **connectors** — the micro-level view that a plain class diagram's aggregation/composition arrows can't express. A part is named `partName : PartType[multiplicity]`. References to objects *outside* the class being modeled are drawn as dashed rectangles.

```
flowchart TD
    subgraph ComputerSystem["«composite» Computer System"]
        PSU[Power Supply Unit]
        Motherboard
        CPU[Central Processing Unit]
        Memory[Memory Module]
        HDD[Hard Disk]
        DVD

        PSU --> Motherboard
        Motherboard --> CPU
        Motherboard --> Memory
        Motherboard --> HDD
        Motherboard --> DVD
    end
```

A **collaboration** is the related concept at the behavior level: it names only the roles and connectors needed

© University of Southampton

7. Profile Diagram

A general extension mechanism for customizing UML for a specific domain (aerospace, medical, finance) or platform (J2EE, .NET), via three mechanisms:

Mechanism	Purpose	Notation
Stereotype	New domain-specific "building block" derived from an existing metaclass	`«stereotypeName»` above the class name
Tag	Extra keyword-value metadata on an element (version, author, test status)	`{key = value}` in curly braces
Constraint	A semantic rule/condition that must always hold	`[constraint text]` in square brackets

A stereotype extends a metaclass (solid arrow, filled head, stereotype → metaclass). To use the stereotypes elsewhere, a package applies the profile (dashed arrow, open head, labeled `«apply»`, pointing from the application package to the profile).

```
classDiagram
    class Component
```

Behavior Diagrams

8. Use Case Diagram

Captures the **functional requirements** of a system as a set of use cases, the actors who trigger them, and the relationships between use cases — without specifying the order of steps (that's what Activity/Sequence diagrams are for). Keep it simple: more than ~20 use cases on one diagram is a sign you're modeling wrong.

Relationships between use cases:

Relationship	Notation	Meaning
Extend	Dashed arrow, open head, → base use case, labeled `«extend»`	Optional behavior triggered under a specific condition (e.g. *Invalid Password* extends *Login*)
Include	Dashed arrow, open head, → included use case, labeled `«include»`	Mandatory reused behavior factored out of the base use case (e.g. *Request Schedule* includes *Search*)
		Child use case inherits and can override the parent's

© University of Southampton

9. Activity Diagram

An extended flowchart modeling **workflow**: how activities/actions are coordinated, including choice, iteration, and concurrency.

Node	Meaning
Initial node	Starts the flow
Action	A task to perform
Decision node	Tests a condition, routes down one path
Merge node	Rejoins paths that split at a decision
Fork node	Splits into parallel/concurrent flows
Join node	Combines parallel flows back together
Object node	An object produced/consumed by the flow (optional)
Final node	Terminates all flow
Swimlane	Groups activities by the actor responsible for them

© University of Southampton

10. State Machine Diagram

Describes an object's **state-dependent behavior**: the states it can be in, the events that trigger transitions, and the actions those transitions perform. Based on Harel statecharts. Used alongside sequence diagrams: a state diagram covers *all* events/states/transitions for *one* object; a sequence diagram covers *one* interaction across *all* objects.

Event types: **signal** (async message arrives), **call** (procedural operation invoked), **time** (a duration elapses), **change** (a watched condition becomes true).

A **transition** fires when its event occurs *and* its guard condition holds; an **action** executes on the transition (not interruptible). A **composite state** holds **substates**, which can nest; a filled circle marks the *initial* state, a circle-with-ring marks the *final* state, and a circled `H` marks a **history** state (re-enter whichever substate was active when the composite was last exited).

Simple transitions (a heating controller):

```
stateDiagram-v2
    [*] --> Idle
    Idle --> CallingStartup : too_hot
    CallingStartup --> CallingReady : compressor_start
    CallingReady --> CallingRun : fan_run
```


© University of Southampton

11. Sequence Diagram

Shows how objects/actors interact **over time** for one scenario: participants laid out horizontally (**lifelines**), time flowing down the page. A thin rectangle on a lifeline (**focus of control** / activation) marks the span during which that element is executing. Arrow styles: filled-head solid = **synchronous** message (caller waits), open-head solid = **asynchronous** (caller continues), dashed = **return**, arrow to self = **self-message**, arrow into a new box = **create**, large `X` = **destroy**.

Combined fragments (UML 2.0) wrap part of the diagram in a labeled box:

Operator	Meaning
`alt`	If/else — mutually exclusive alternatives
`opt`	Optional block, runs only if its condition holds
`par`	Parallel — fragments execute concurrently
`loop`	Repeats while a condition holds
`break`	Early exit from an enclosing fragment
`critical`	Must run uninterrupted by other messages

© University of Southampton

12. Communication Diagram

Semantically equivalent to a sequence diagram (either can be derived from the other) — the difference is purely organizational: a **sequence diagram** arranges messages by **time**; a **communication diagram** arranges objects by **space/relationship** and numbers the messages instead (`1`, `1.1`, `1.2`, `2`, …) to show order. Best when the thing worth seeing is *who talks to whom*, not *when*.

```
flowchart LR
    Student -->|"1: returnBook()"| Librarian
    Librarian -->|"1.1: checkDueDate()"| Catalog[(Book Record)]
    Librarian -->|"1.2: calculateFine()"| Catalog
    Librarian -->|"2: confirmReturn()"| Student
```

Objects are either **suppliers** (provide the called method, receive the message) or **clients** (call the method, send the message). A message an object sends to itself is drawn as a self-loop, same as in a sequence diagram.

© University of Southampton

13. Interaction Overview Diagram

A high-level **activity diagram whose nodes are interactions** (small sequence diagrams, or `ref` pointers to interactions defined elsewhere) instead of plain actions — useful for stitching several sequence diagrams into one controlled flow without redrawing them inline.

```
flowchart TD
    Start((•)) --> Offer{Accept offer?}
    Offer -->|reject| EndNode((•))
    Offer -->|accept| RegisterHR[/"ref: Register with HR"/]
    RegisterHR --> ApplyCar[/"ref: Apply for Company Car"/]
    ApplyCar --> Onboard[/"ref: Onboard Customers"/]
    Onboard --> Probation{Completed probation?}
    Probation -->|no| Terminate[Contract Terminated]
    Probation -->|yes| EndNode
    Terminate --> EndNode
```

Employee onboarding: rejecting the offer skips straight to the end; accepting routes through three referenced interactions before the probation decision.

© University of Southampton

14. Timing Diagram

A specialized sequence diagram for showing **when** state or value changes happen, not just their relative order. Lifelines run horizontally (left → right = elapsed time); a **state lifeline** plots which state the object is in at each moment, a **value lifeline** plots a changing value — both change level only when the underlying state/value actually changes. Must stay consistent with the matching sequence/state diagrams for the same object.

```
flowchart LR
    Closed -->|"car arrives (t=0s)"| Opening
    Opening -->|"t=2s"| Open
    Open -->|"car passes (t=5s)"| Closing
    Closing -->|"t=7s"| Closed
```

A car-park gate's state over time — the same object modeled as a state machine diagram, just read along a time axis instead of as a graph of transitions.

© University of Southampton